



E N G E N H A R I A   D E   D A D O S

# BÚSSOLA PÚBLICA

## Pipeline Radar Legislativo

D O C U M E N T A Ç Ã O   T É C N I C A   |   D A T A   C H A L L E N G E

*Integração Python, OpenAI, Supabase e n8n*

---

**DROP DATABASE**

SQUAD 21001 · DATA ENGINEERING SQUAD

Data Challenger · Pós Tech Xperiun · Projeto "Bússola Pública" (DataChallenges) · Entrega Final

# Sumário

---

|                                                       |    |
|-------------------------------------------------------|----|
| 1. Introdução e Cenário de Negócio                    | 3  |
| 2. Arquitetura de Dados e Pipeline Estrutural         | 3  |
| 3. Dicionário de Dados — Tabela dim_proposicoes       | 5  |
| 4. Orquestração e Resiliência Estrutural (n8n)        | 6  |
| 5. Execução Local e Comandos Essenciais               | 7  |
| 6. Esquema do Banco de Dados (Star Schema + pgvector) | 8  |
| 7. Prompt de Resumo Executivo de 3 Linhas             | 8  |
| 8. Justificativas Arquiteturais                       | 9  |
| 9. Conclusão e Próximos Passos                        | 10 |

# 1. Introdução e Cenário de Negócio

## 1.1 Problema de Negócio

A consultoria fictícia Bússola Pública monitora, para clientes de Relações Governamentais (GR), a atividade legislativa da Câmara dos Deputados — proposições, votações, despesas parlamentares e composição partidária. O acompanhamento manual desse volume de informações é inviável: dados relevantes para a tomada de decisão ficam dispersos, são identificados tardiamente, e não há padronização na priorização de temas estratégicos para cada cliente.

O SQUAD 21001 endereçou esse problema construindo um pipeline de dados automatizado, seguindo a Arquitetura Medalhão (Bronze → Silver → Gold), que extrai, limpa, modela e enriquece com Inteligência Artificial os dados públicos da Câmara dos Deputados, disponibilizando-os em um Data Warehouse na nuvem (Supabase/PostgreSQL) e notificando automaticamente, via n8n, novas proposições relevantes — classificadas por tema estratégico e resumidas executivamente em 3 linhas.

## 1.2 Benefícios Entregues

| RESUMO   Benefícios da Solução |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
|--------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Racional                       | O acompanhamento manual de proposições, votações e despesas é lento, disperso e sujeito a falhas humanas, atrasando a resposta de clientes de GR a temas estratégicos.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
| Objetivo                       | Automatizar a extração, modelagem, enriquecimento por IA e distribuição de informações legislativas relevantes, com priorização temática automática.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| Insight Esperado               | Centralizar dados em um Star Schema na nuvem, com camada de IA capaz de classificar temas e gerar resumos executivos sem intervenção manual.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
| Resultado Obtido               | <ul style="list-style-type: none"><li>• Redução do tempo de monitoramento manual de proposições legislativas.</li><li>• Priorização automática por tema estratégico, via classificação por similaridade de embeddings de IA.</li><li>• Resumos executivos de 3 linhas, prontos para tomada de decisão por clientes corporativos.</li><li>• Notificações diárias automáticas (e-mail e Telegram) sobre novas proposições relevantes.</li><li>• Base estruturada em Star Schema, pronta para análises históricas e dashboards futuros.</li><li>• Camada vetorial (pgvector) preparada para busca semântica e RAG sobre o histórico de proposições.</li></ul> |

# 2. Arquitetura de Dados e Pipeline Estrutural

O pipeline da entrega final segue a Arquitetura Medalhão, orquestrada pelo módulo `main.py`, com cinco macroetapas (extração, transformação, conversão para Gold/Parquet, enriquecimento por IA e carga no Supabase), seguidas de uma camada de orquestração e notificação via n8n.

## Bloco 1 | Camadas do Pipeline (Medalhão)

As três camadas a seguir documentam o fluxo de dados desde a ingestão bruta até o enriquecimento cognitivo e a persistência final.

| E01   Camada de Ingestão e Processamento Local (Python 3.12 + uv) |                                                                                                                                                                                                                                                                       |
|-------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Racional                                                          | É necessário extrair dados brutos e heterogêneos da API de Dados Abertos da Câmara e padronizá-los antes de qualquer análise ou carga em nuvem.                                                                                                                       |
| Objetivo                                                          | Implementar módulos <code>src/extract_*.py</code> e <code>src/transform_*.py</code> que consomem a API (deputados, partidos, proposições, votações e despesas) e geram as camadas Bronze e Silver.                                                                    |
| Insight Esperado                                                  | A conversão para Apache Parquet (compressão Snappy, via PyArrow) reduziria custo de armazenamento e melhoraria a performance de leitura frente ao CSV tradicional.                                                                                                    |
| Resultado Obtido                                                  | A engine local em Python 3.12, orquestrada pelo uv, salva os dados brutos em <code>data/raw/</code> (Bronze), aplica limpeza com Pandas gerando <code>data/silver/</code> , e <code>src/gold_stage.py</code> converte as 5 tabelas Silver para Parquet (camada Gold). |

| E02   Camada de Inteligência Artificial Extrativa (OpenAI) |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
|------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Racional                                                   | Ementas de proposições são extensas e redigidas em linguagem jurídica, dificultando a priorização rápida por clientes de GR.                                                                                                                                                                                                                                                                                                                                                             |
| Objetivo                                                   | Submeter os dados consolidados em Parquet ao módulo <code>src/enrich_proposicoes_ia.py</code> , executando dois passos cognitivos via API da OpenAI.                                                                                                                                                                                                                                                                                                                                     |
| Insight Esperado                                           | <ol style="list-style-type: none"> <li>Classificação Temática Automatizada: cada proposição classificada em um dos 10 temas macro-estratégicos (TEMAS_ALVO) por similaridade de cosseno entre o embedding da ementa (<code>text-embedding-3-small</code>) e o embedding de cada tema.</li> <li>Resumo Executivo de Alta Densidade: tradução da linguagem jurídica complexa em um resumo analítico de até 3 linhas, voltado a tomadores de decisão (<code>gpt-4o-mini</code>).</li> </ol> |
| Resultado Obtido                                           | Por controle de custo, a execução de validação processa um lote inicial controlado de proposições, gravando o resultado em <code>data/gold/dim_proposicoes_ia.csv</code> .                                                                                                                                                                                                                                                                                                               |

| E03   Camada de Armazenamento Centralizado (Supabase) |                                                                                                                                                                                |
|-------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Racional                                              | Os dados enriquecidos precisam estar disponíveis em um Data Warehouse na nuvem, acessível a dashboards, à camada vetorial e ao workflow n8n.                                   |
| Objetivo                                              | Persistir o payload processado em um banco PostgreSQL hospedado no Supabase, por meio de <code>src/load_to_supabase.py</code> .                                                |
| Insight Esperado                                      | O uso de SQLAlchemy com pool de conexões e carga incremental em lote ( <code>to_sql</code> , chunked) suportaria o volume das tabelas do Star Schema sem degradar performance. |
| Resultado Obtido                                      | Carga incremental em lote operacional para todas as tabelas do Star Schema, com pool de conexões gerenciado via SQLAlchemy.                                                    |

### 3. Dicionário de Dados — Tabela dim\_proposicoes

A tabela **dim\_proposicoes** centraliza os metadados regulatórios e os campos analíticos gerados pela camada de Inteligência Artificial. É o núcleo da entrega final, pois conecta os dados brutos da API da Câmara aos insights gerados por IA.

| Campo               | Tipo         | Restrição   | Descrição                                                                              |
|---------------------|--------------|-------------|----------------------------------------------------------------------------------------|
| ID_Proposicao       | INT8         | PRIMARY KEY | Identificador único numérico gerado pela Câmara dos Deputados.                         |
| Tipo_Proposicao     | VARCHAR(10)  | NOT NULL    | Sigla da proposição legislativa (PL, PEC, MPV, MSC etc.).                              |
| Numero_Proposicao   | INT4         | NOT NULL    | Número identificador oficial do projeto de lei.                                        |
| Ano_Proposicao      | INT4         | NOT NULL    | Ano de início da tramitação legislativa.                                               |
| Ementa              | TEXT         | NULLABLE    | Texto descritivo original detalhando o escopo do projeto.                              |
| Tema_IA             | VARCHAR(100) | NULLABLE    | Macrotema estratégico atribuído dinamicamente via similaridade de embeddings (OpenAI). |
| Resumo_Executivo_IA | TEXT         | NULLABLE    | Resumo executivo de 3 linhas gerado por IA (gpt-4o-mini).                              |
| dataApresentacao    | DATE         | NOT NULL    | Data oficial de protocolo da proposição legislativa.                                   |

A tabela **proposicoes\_embeddings** (camada vetorial) referencia **dim\_proposicoes** via FK **id\_proposicao** e armazena, para cada modelo de embedding, o vetor (**VECTOR(1536)**) com índice HNSW para busca por similaridade — ver Seção 6 para o DDL completo.

#### 3.1 Visão Geral do Star Schema

| Tabela                 | Descrição                                                                                                                                                                              |
|------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| dim_partidos           | Dimensão de partidos políticos com representação na Câmara dos Deputados.                                                                                                              |
| dim_deputados          | Dimensão de deputados em exercício, com FK para <b>dim_partidos</b> .                                                                                                                  |
| dim_proposicoes        | Dimensão de proposições legislativas, incluindo os campos <b>Tema_IA</b> e <b>Resumo_Executivo_IA</b> gerados pela camada de IA (ver Seção 2).                                         |
| fato_votacoes          | Fatos de votações realizadas no plenário e nas comissões, com FK para <b>dim_proposicoes</b> .                                                                                         |
| fato_despesas          | Fatos de despesas parlamentares (CEAP), com FK para <b>dim_deputados</b> .                                                                                                             |
| proposicoes_embeddings | Tabela vetorial dedicada (BIGSERIAL PK, FK para <b>dim_proposicoes</b> , <b>embedding VECTOR(1536)</b> , <b>UNIQUE(id_proposicao, modelo)</b> ), com índice HNSW para busca semântica. |

## 4. Orquestração e Resiliência Estrutural (n8n)

A camada de entrega e orquestração baseia-se na plataforma n8n, responsável por executar de forma cronometrada a lógica de negócio e os disparos de notificações automáticas. O workflow é composto por 8 nós, executados diariamente às 06h:

| Nó                                          | Função                                                                                                                                         |
|---------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------|
| Execução Diária 06h (Schedule Trigger)      | Dispara o fluxo automaticamente todos os dias às 06h00 (UTC-3).                                                                                |
| Set Config                                  | Centraliza parâmetros do Squad e monta a expressão de filtro PostgREST a partir dos temas monitorados ( <code>in. ("Tema1", "Tema2")</code> ). |
| Busca Novas Proposições - IA (HTTP Request) | Consulta <code>dim_proposicoes</code> via PostgREST do Supabase, com a política "Always Output Data" ativada.                                  |
| Agrupar Lote de Proposições (Aggregate)     | Consolida os itens retornados em um único lote para formatação das mensagens.                                                                  |
| Formata Email e Telegram (Code/JS)          | Normaliza o payload e monta o HTML do e-mail e o Markdown do Telegram, tratando o cenário de lista vazia.                                      |
| Envia Email (Gmail)                         | Envia a notificação diária em template HTML estilizado para os clientes do Squad.                                                              |
| Envia Telegram (Telegram)                   | Envia mensagem em Markdown ao bot configurado para o canal do Squad.                                                                           |

### Bloco 2 | Decisões de Resiliência e Distribuição

Os três pontos a seguir documentam decisões de engenharia que garantem estabilidade e tolerância a falhas no workflow diário.

| N01   Tratamento Inteligente de Filtros de Texto na API REST |                                                                                                                                                    |
|--------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------|
| Racional                                                     | A consulta ao Supabase via PostgREST instabilizava ao lidar com strings contendo espaços (ex.: "Educação e Cultura"), gerando erros de requisição. |
| Objetivo                                                     | Garantir o escape correto de caracteres especiais na URL e prevenir falhas HTTP 400 Bad Request.                                                   |
| Insight Esperado                                             | A formatação dinâmica de texto poderia ser centralizada no próprio nó Set Config, sem necessidade de tratamento adicional posterior.               |
| Resultado Obtido                                             | O nó Set Config implementa formatação dinâmica de texto, eliminando erros HTTP 400 nas consultas PostgREST.                                        |

| N02   Política de Resiliência — "Never Drop Items" |                                                                                                                                                                           |
|----------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Racional                                           | Quando a fonte de dados retorna lista vazia ( <code>[]</code> ), o workflow assíncrono poderia ser abortado, interrompendo a notificação diária.                          |
| Objetivo                                           | Garantir que o cliente do Squad receba sempre a notificação diária, mesmo sem novas proposições.                                                                          |
| Insight Esperado                                   | Parametrizar o nó HTTP "Busca Novas Proposições - IA" com a política "Always Output Data" (never drop items) evitaria a interrupção do fluxo.                             |
| Resultado Obtido                                   | Mesmo sem novas proposições, o workflow não é abortado: o nó subsequente identifica o cenário de "Zero Proposições" e gera uma mensagem padronizada de monitoria estável. |

| N03   Camada de Distribuição e Normalização |                                                                                                                                                                                                                                                                                                                                                                                |
|---------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Racional                                    | O payload pode variar conforme o cenário de execução (modo isolado vs. modo integrado ao workflow), exigindo normalização antes da distribuição.                                                                                                                                                                                                                               |
| Objetivo                                    | Normalizar o payload via nó de código (JavaScript) e gerar duas saídas paralelas de notificação.                                                                                                                                                                                                                                                                               |
| Insight Esperado                            | Um único nó de normalização poderia alimentar simultaneamente os canais de e-mail e Telegram, com formatações distintas.                                                                                                                                                                                                                                                       |
| Resultado Obtido                            | <ul style="list-style-type: none"><li>• <b>Gmail:</b> envio de template HTML corporativo, com estilos inline, contendo o <code>Tema_IA</code> e o <code>Resumo_Executivo_IA</code> de cada nova proposição.</li><li>• <b>Telegram:</b> mensagens formatadas em Markdown, com limite automático de caracteres para evitar estouro de buffers visuais em telas móveis.</li></ul> |

## 5. Execução Local e Comandos Essenciais

Pré-requisitos: Python 3.12+, gerenciador de pacotes uv, conta Supabase com extensão pgvector habilitada, e variáveis de ambiente configuradas em um arquivo `.env` local (não versionado), incluindo `DB_URL`, `OPENAI_API_KEY` e demais chaves do projeto.

### 5.1 Configuração do Ambiente

```
# clonar o repositório do projeto
git clone <url-do-repositorio>
cd DataChallenges

# criar e ativar ambiente virtual com uv
uv venv
source .venv/bin/activate # Linux/Mac
.venv\Scripts\activate    # Windows

# instalar dependências
uv pip install -r requirements.txt

# configurar variáveis de ambiente
cp .env.example .env # preencher DB_URL, OPENAI_API_KEY, etc.
```

### 5.2 Comandos de Execução

```
# pipeline completo (Bronze -> Silver -> Gold -> IA -> Supabase)
python main.py

# validar conexão com o Supabase
python src/database.py

# executar a extensão vetorial e o schema completo (uma vez)
psql "$DB_URL" -f supabase_schema.sql

# ingestão incremental diária (últimos 30 dias) - antes do trigger n8n das 06h
python ingestion_script.py
```

### 5.3 Dependências Principais

- **pandas**, **pyarrow** — manipulação e gravação Parquet (camada Gold).
- **requests** — extração via API de Dados Abertos da Câmara dos Deputados.
- **openai** — geração de embeddings (text-embedding-3-small) e resumos executivos (gpt-4o-mini).
- **sqlalchemy**, **psycopg2** — conexão e carga no PostgreSQL/Supabase.
- **python-dotenv** — carregamento de variáveis de ambiente a partir do arquivo `.env`.

## 6. Esquema do Banco de Dados (Star Schema + pgvector)

O DDL completo está disponível em [supabase\\_schema.sql](#). Resumo das tabelas que compõem o modelo:

- **dim\_partidos** — dimensão de partidos políticos.
- **dim\_deputados** — dimensão de deputados, com FK para **dim\_partidos**.
- **dim\_proposicoes** — dimensão de proposições, incluindo **Tema\_IA** e **Resumo\_Executivo\_IA** (ver Seção 3).
- **fato\_despesas** — fatos de despesas parlamentares, com FK para **dim\_deputados**.
- **fato\_votacoes** — fatos de votações, com FK para **dim\_proposicoes**.
- **proposicoes\_embeddings** — tabela vetorial dedicada, com FK para **dim\_proposicoes**, coluna **embedding VECTOR(1536)** e índice HNSW (**vector\_cosine\_ops**).

### 6.1 DDL — Camada Vetorial (proposicoes\_embeddings)

```
create extension if not exists vector;

create table proposicoes_embeddings (
  id bigserial primary key,
  id_proposicao bigint references dim_proposicoes(id_proposicao),
  modelo varchar(50) not null,
  embedding vector(1536) not null,
  unique (id_proposicao, modelo)
);

create index idx_proposicoes_embeddings_hnsw
on proposicoes_embeddings
using hnsw (embedding vector_cosine_ops)
with (m = 16, ef_construction = 64);
```

## 7. Prompt de Resumo Executivo de 3 Linhas

Prompt oficial padronizado do projeto, utilizado para gerar o campo **Resumo\_Executivo\_IA**:

"Resuma em 3 linhas, com tom executivo e direcionado a tomadores de decisão, o conteúdo abaixo. Foco nas implicações práticas e no que é essencial para ação imediata: {texto\_da\_proposicao}. Resuma em 3 linhas."

### 7.1 Nota de Implementação

O código atual em [src/enrich\\_proposicoes\\_ia.py](#) (função **gerar\_resumo\_executivo**) usa uma variação funcionalmente equivalente, com persona de "analista sênior de Relações Governamentais" e foco em impacto prático para empresas, negócios ou cidadãos. Recomenda-se padronizar essa função para utilizar o prompt oficial acima, garantindo consistência entre o pipeline completo ([main.py](#)) e a ingestão incremental ([ingestion\\_script.py](#)).



## 8. Justificativas Arquiteturais

### Bloco 3 | Decisões de Modelagem e Custo

As três decisões a seguir documentam os principais trade-offs de modelagem, performance e custo do projeto.

| J01   Tabela Vetorial Dedicada com FK |                                                                                                                                                                                                                                                 |
|---------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Racional                              | Embeddings (1536 floats) raramente são lidos em consultas analíticas comuns e, se embutidos em <code>dim_proposicoes</code> , inflariam a tabela dimensional principal.                                                                         |
| Objetivo                              | Preservar o desempenho de varreduras e índices convencionais sobre <code>dim_proposicoes</code> , isolando o índice HNSW (custo de manutenção próprio) em estrutura dedicada.                                                                   |
| Insight Esperado                      | Uma tabela <code>proposicoes_embeddings</code> separada, ligada por FK, permitiria versionar múltiplos embeddings por proposição (coluna <code>modelo</code> , com <code>UNIQUE(id_proposicao, modelo)</code> ) sem alterar a tabela principal. |
| Resultado Obtido                      | Separação implementada: <code>proposicoes_embeddings</code> isola os vetores e o índice HNSW, mantendo <code>dim_proposicoes</code> performática e preparada para migração futura de modelo de embedding.                                       |

| J02   pgvector + Índice HNSW como Busca Semântica em Escala |                                                                                                                                                                        |
|-------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Racional                                                    | Manter um banco vetorial externo adicionaria complexidade operacional ao stack já baseado em PostgreSQL/ Supabase.                                                     |
| Objetivo                                                    | Integrar a busca vetorial diretamente ao Data Warehouse existente, com bom equilíbrio entre velocidade de consulta e custo de construção/atualização.                  |
| Insight Esperado                                            | O índice HNSW (Hierarchical Navigable Small World) ofereceria busca aproximada por vizinhos mais próximos com latência sublinear mesmo em volumes elevados de vetores. |
| Resultado Obtido                                            | pgvector + HNSW adotado como solução nativa, adequado ao volume atual e ao crescimento esperado do projeto, sem banco vetorial externo.                                |

| J03   Ingestão Incremental dos Últimos 30 Dias |                                                                                                                                                                                                                                           |
|------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Racional                                       | Reprocessar diariamente o histórico completo de proposições seria custoso: cada chamada às APIs de embeddings ( <code>text-embedding-3-small</code> ) e de resumo ( <code>gpt-4o-mini</code> ) tem custo monetário e latência associados. |
| Objetivo                                       | Capturar proposições novas ou recentemente atualizadas sem comprometer a janela de orquestração diária do n8n (06h).                                                                                                                      |
| Insight Esperado                               | Uma janela de 30 dias seria suficiente para garantir atualidade dos dados monitorados, reduzindo drasticamente o custo de chamadas a APIs externas.                                                                                       |
| Resultado Obtido                               | Ingestão incremental de 30 dias implementada em <code>ingestion_script.py</code> , mantendo tempo de execução compatível com a janela diária do n8n.                                                                                      |

## 9. Conclusão e Próximos Passos

---

O pipeline Bússola Pública — Radar Legislativo, entregue pelo SQUAD 21001, está estruturado de ponta a ponta: extração, transformação, conversão para Parquet, enriquecimento por IA generativa, carga em Star Schema com camada vetorial e notificação automatizada via n8n. A integração entre as camadas confere maturidade de nível de produção ao projeto, com documentação, scripts de ingestão e schema de banco prontos para uso.

### 9.1 Próximos Passos (Expansão Futura)

- Camada de marketing político: novas tabelas de fatos e dimensões para análise de sentimento em redes sociais e emendas parlamentares.
- Uso da camada vetorial (pgvector) para busca semântica e RAG sobre o histórico de proposições.
- Agendamento do `ingestion_script.py` via GitHub Actions, em rotina serverless executada antes do trigger diário do n8n.
- Monitoramento de logs centralizado para volumetrias extremas de dados históricos.
- Dashboards analíticos sobre o Star Schema consolidado no Supabase.
- Padronização do prompt de resumo executivo entre `main.py` e `ingestion_script.py`.